

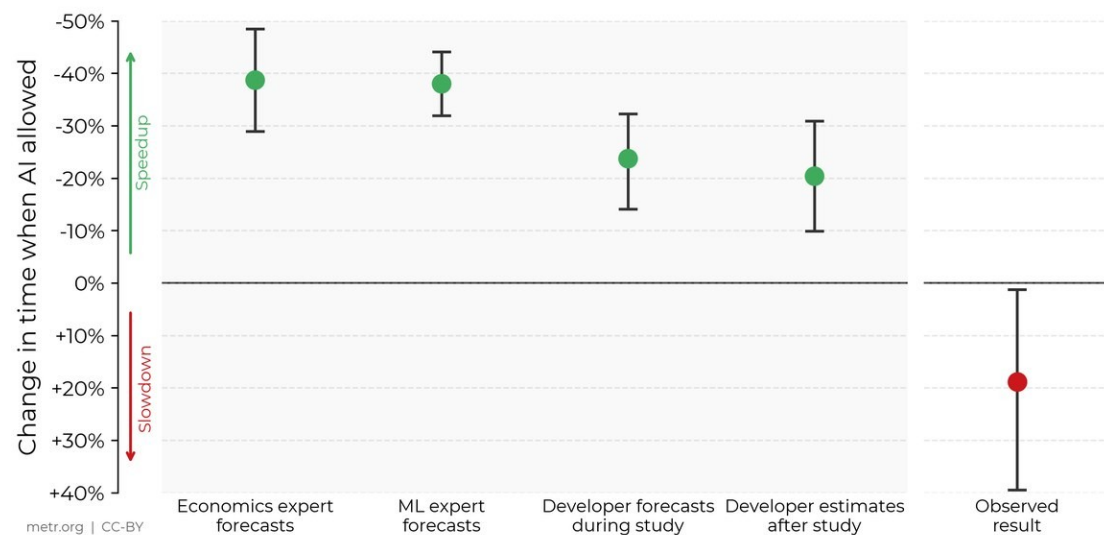
16 экспертов были уверены, что AI их ускоряет — и ошиблись на знак



Against Expert Forecasts and Developer Self-Reports, Early-2025 AI Slows Down Experienced Open-Source Developers



In this RCT, 16 developers with moderate AI experience complete 246 tasks in large and complex projects on which they have an average of 5 years of prior experience.



Прогноз –24% · вера –20% · факт +19% времени

Эксперимент METR, первая половина 2025

16 опытных разработчиков открытого кода · 246 реальных задач в своих знакомых репозиториях; измеряли реальное время, не ощущение

Три числа об одном и том же

Прогноз до эксперимента: AI ускорит на –24%

Вера после работы: ускорил примерно на –20%

Измеренный факт: с AI задачи заняли на +19% дольше

— METR (Model Evaluation and Threat Research, независимая исследовательская организация), RCT (randomized controlled trial, рандомизированный контролируемый эксперимент), n=16, 246 задач, 2025

Профессионалы ошиблись не в величине — в знаке. «Мне кажется, инструмент помогает» — гипотеза, а не данные; надёжность даёт дисциплина с встроенной проверкой, а не ощущение. ^[1]

[1] METR — RCT: AI и опытные OSS-разработчики (+19% времени)

04

ЛЕКЦИЯ 4

AI в жизненном цикле разработки ПО

Курс «Осознанное применение AI» · инженеры-разработчики



Надёжность даёт не инструмент, а инженерная дисциплина по фазам: какой артефакт и какая проверка нужны на каждой.

0
Введение

1
Требования

2
Архитектура

3
Реализация

4
Тестирование

5
Ревью+Безоп.

6
Доставка+

7
Обобщение

Стоим на фундаменте Модуля 1 — берём одну индустрию и разбираем по фазам



Из Лекции 1

Слоистая картина «модель → чат → агент → приложение»; промпт как «роль + задача + контекст». Пользуемся как словарём.



Из Лекции 2

Почему AI выдаёт «почти правильный» текст: ответ генерируется потоково, поэтому правдоподобный ≠ верный. Одной фразой.



Из Лекции 3

Лестница сложности («оставайся на нижней ступени») + критерий «когда не ИИ вовсе» — детерминированную задачу решает обычный код.



Из Лекции 3

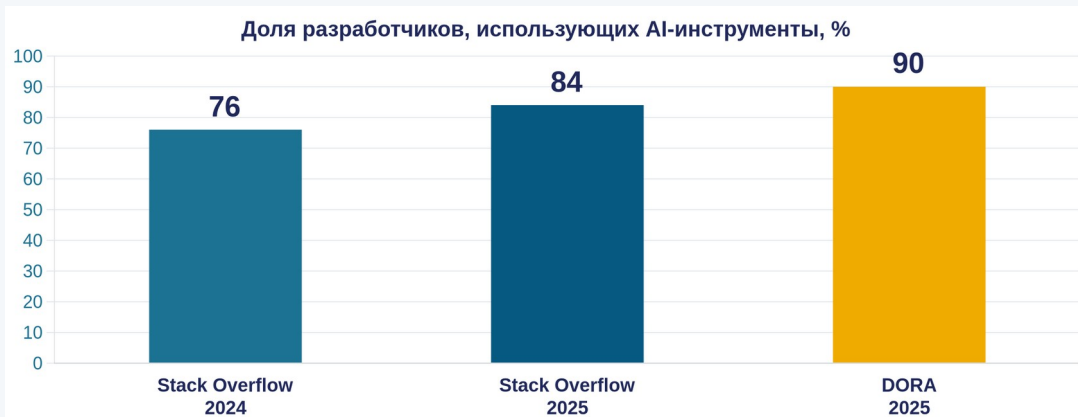
Цикл агента plan → act → check → iterate (4 места отказа) + prompt injection (защита архитектурная, не фильтрацией).

Это переносим готовым и не переобъясняем — только применяем к разработке ПО. Модуль 1 дал аппарат выбора архитектуры; Лекция 4 показывает, какая дисциплина делает AI в одной индустрии надёжным по фазам — и где ломается без неё.

Вопрос «применять или нет» индустрия уже закрыла — открытым остался вопрос надёжности



Масштаб: почти вся отрасль уже внутри



51% профессиональных разработчиков работают с AI-инструментами ежедневно; медиана времени работы с ними — около двух часов в день, примерно четверть рабочего дня.

Опросы разные: Stack Overflow спрашивает «используют или планируют» (более 49 000 человек из 177 стран), DORA — «используют» (более 5 000). Порядок величины совпадает.



Доверие при этом падает

46%

не доверяют точности того, что выдают AI-инструменты — против 31% годом ранее ^[1]

Более 80% при этом отмечают рост личной эффективности — а высоко доверяют результату лишь 24% ^[2]. Ощущение пользы и доверие к выводу разошлись.

Пользуются почти все — доверяют меньше половины. Спорить «включать ли AI» уже не о чем; инженерный вопрос — чем закрыть этот разрыв.

84-90% отрасли уже внутри, и доля растёт второй год подряд. Значит полезный вопрос не «применять AI или нет», а какой ценой, с какими рисками и какими мерами они закрываются — по фазам разработки.

Вопрос лекции — про дисциплину, а не про инструмент

AI пишет код всё лучше — но что делает AI-разработку надёжной?

Не инструмент, а инженерная дисциплина по фазам: какой человеко-владеемый артефакт и какая проверка нужны на каждой фазе, где методики разных игроков сходятся — и что не делегируется инструменту ни в одной фазе. ^[1]

Какой инструмент лучший?

Устаревает за квартал; «лучший» невозможно назвать без «для чего, в какой фазе, в каком режиме».



Какая практика оправдана в фазе?

Каким артефактом заканчивается? Где обязателен человек? — устойчиво годами.

Инструмент вторичен, практика первична. Отвечая, инженер называет не логотип, а уместную практику фазы, её артефакт и точку обязательного человеческого контроля — знание, которое переживёт смену любого вендора. ^[2]

Эта лекция — сводка практик: современные подходы лидеров + проверенная классика



А. Современные практики (лидеры)

[1] [Anthropic — Claude Code / AI-Native SDLC](#)

агентный git-цикл: каждый этап коммитит артефакт; человек на гейтах

[2] [OpenAI — Model Spec](#)

спека-как-контракт; клауза = пример-промпт = тест

[3] [GitHub — Spec Kit](#)

«намерение — источник истины»; малые проверяемые задачи

[4] [Google — DORA 2025](#)

«AI усиливает то, что уже есть»; семь способностей

[5] [Thoughtworks — Exploring Gen AI](#)

ассистент предлагает, разработчик владеет; harness engineering

[6] [Willison — Vibe engineering](#)

дисциплины, которые LLM вознаграждает: тесты, планы, ревью



В. Проверенная временем классика

[7] [Brooks — No Silver Bullet](#)

существенная vs привнесённая сложность; «что строить» — человеку

[8] [Beck — TDD: By Example](#)

red-green-refactor; тест-как-спецификация

[9] [Nygard — ADR](#)

неизменяемые записи «почему» в контроле версий

[10] [Ford/Parsons — Evolutionary Architectures](#)

fitness-функции: «пригодность» объективна и автоматична

[11] [Fowler — Refactoring](#)

дисциплина малых проверяемых изменений

[12] [Brown — C4](#)

архитектура-как-код: текстовая диффабельная модель для AI

Инструменты сменяются за кварталы — эти практики устойчивы: опираются на природу сложности, а не на зрелость продукта. Все названия — кликабельные ссылки на первоисточники.

Дисциплина — это цикл фаз, и каждой фазой владеет человек через свой артефакт



Фаза — стадия со своим входом / выходом / артефактом.

AI работает ВНУТРИ узлов (набрасывает требования, предлагает ADR, пишет код в PR), но владеет каждым узлом человек — читает, правит, принимает, отвечает ^[1].

Практики зрелы неравномерно

сильные: требования, реализация, тестирование,
ревью · тонкие: архитектура, доставка, эксплуатация
· светлый пятачок: документация.

На этом скелете фаз независимо сошлись Anthropic ^[1], OpenAI ^[2], DORA ^[3], Thoughtworks ^[4] — значит перед нами методика, а не мода.

Автономия — это свойство режима, а не бренда

Лестница автономности — вспомогательная линза

A

автодополнение

дописывает строку; человек принимает каждое

B

мелкие задачи

функция/фикс в диалоге; человек ревьюит после

C

кодинг-агент

планирует, правит файлы, гоняет тесты; человек — merge

D

оркестратор

берёт задачу из трекера → PR; человек — прод-гейт

Режим ≠ бренд — один продукт живёт сразу на нескольких ступенях ^[2]

Copilot — A, B, C и D · Cursor — Tab (A), Cmd-K (B), Composer (C) · Claude Code — C, поднимается до D

B ↔ C итерирует ли сам и гоняет ли тесты без вас (да → C)

C ↔ D откуда задача и куда результат (из трекера → PR → D)

Чем выше автономия, тем строже критерий «здесь обязателен человек» и жёстче харнес. «Мы используем Copilot» не сообщает ни уровень, ни фазу — называем режим и фазу, а не логотип. ^[1]

Методика определяет, инструмент исполняет

Методическая практика — решение о том, какой артефакт произвести, в каком порядке, с какой проверкой и кто отвечает. Инструмент — исполнитель: взаимозаменяемый и вторичный.



Волатильность

Инструменты и их «лидерство» меняются за кварталы; практики устойчивы годами — упираются в природу сложности (Brooks ^[1]), а не в зрелость продукта.



Множитель DORA

«AI усиливает то, что уже есть» ^[2]:
первично построить дисциплину, а не выбрать инструмент — инструмент без дисциплины умножает хаос.



Ответственность

Инструмент не отвечает за последствия, отвечает человек; ответственность материализуется в артефактах и гейтах, то есть в практике.

«Мы внедрили AI-инструмент» $\neq$ «мы внедрили AI-дисциплину». Инструмент есть, практики нет — и множитель DORA (DevOps Research and Assessment — исследовательская программа Google по зрелости DevOps-практик) работает в худшую сторону. За этим стоят все провалы лекции: prompt-and-pray, отравленный контекст, 70%-проблема, инцидент Replit — везде инструмент применён без практики.

**Методико-first-порядок: сначала практика фазы (артефакт, гейт, кто отвечает), потом инструмент под неё.
Инструмент выбирается под дисциплину, а не дисциплина под инструмент. ^[3]**

РАЗДЕЛ 1

Требования — первый артефакт, ещё до кода

Первая фаза цикла превращает размытое намерение в требования. Ведущее здесь — не инструмент, а дисциплина требований (spec-driven): между намерением и кодом ставится человеко-ревьюируемый артефакт, а решение «что именно нам нужно» остаётся человеческим.



0

Введение

1

Требования

2

Архитектура

3

Реализация

4

Тестирование

5

Ревью+Безоп.

6

Доставка+

7

Обобщение

Первый артефакт разработки — требования, а не код



**Репозиторий: требования
версионруются рядом с кодом** ^[3]



требования.md (spec.md)

что система должна делать



design.md

ограничения и решения



tasks.md

декомпозиция на задачи



src/...

код — порождается из требований

Порядок принудителен: требования → дизайн → задачи (Kiro / Spec-Kit ^[3]). Дороже всего — ошибки этой фазы.

OpenAI Model Spec ^[2]

живой версионизируемый Markdown: требования — долговечный, ревьюируемый, диффабельный артефакт, а не мимолётные промпты.

Sean Grove, OpenAI · «The New Code» ^[5]

«исходные требования — вот ценный артефакт»; код — «структурированная коммуникация».

Martin Fowler ^[4]

узкое место разработки с AI — намерение: код модель пишет всё лучше, а выразить, что строить, по-прежнему трудно.

Grove «код ~10-20% ценности» — риторическая провокация, а не эмпирическое измерение: её задача сместить фокус на требования.

AI силен в структурировании и полноте намерения; в самом намерении — что нужно системе — решает человек ^[1].

Дисциплина заранее — быстрый результат; без неё отказ ожидаем, не случаен



AWS Kiro · life sciences (фарма/биотех, 2026)

Продакшен-готовый агент для поиска терапевтических мишеней:
рабочий процесс spec-first — спецификация → исполнение по вехам
→ верификация против спеки.

3 недели, 3 разработчика

— не растянутый цикл *ad-hoc*-промптинга.

Средняя достоверность — опубликованный вендором кейс-стади, не независимый аудит.



847 задокументированных внедрений AI-агентов

76% отказали за 90 дней

Корневая причина по источнику — провал спецификации, не техническая неспособность модели.

Намерение не было специфицировано настолько точно, чтобы его вообще можно было проверить.

Низкая-средняя достоверность — методология выборки не раскрыта; иллюстрация тезиса, не аудированная статистика.

SDD (Spec-Driven Development, разработка через спецификацию) — GitHub Spec Kit ~90 тыс. звёзд; к 2026 — минимум 8 крупных вендоров с собственным вариантом практики.

Дисциплина заранее — не гарантия, а сдвиг вероятности: спецификация намерения делает отказ исключением, а не статистически ожидаемым исходом.

Как вести требования: структура (как писать) + процесс (как поддерживать)



СТРУКТУРА — как писать требования

- **Истории + критерии приёмки** ^[1]

«Как <роль>, я хочу <цель>, чтобы <польза>» + проверяемые критерии на каждую историю.

- **EARS-нотация (лёгкий синтаксис требований)** ^[7]

«КОГДА <триггер>, система ДОЛЖНА <ответ>» (Mavin 2009) — убирает «should/may», делает требование testable.

- **Функциональные vs нефункциональные** ^[6]

поведение отдельно от характеристик (latency / cost / security); NFR (нефункциональные требования) энфорсятся fitness-функциями.

- **требования → дизайн → задачи** ^[3]

принудительный порядок 3 файлов (Kiro / Spec-Kit); DoD — малые изолированно-тестируемые единицы.



ПРОЦЕСС — как поддерживать требования

- **Выявление: интервьюирующий LLM** ^[4]

модель ЗАДАЁТ вопросы (Fowler «Interrogatory LLM»), вскрывая невысказанные допущения — а не «промт-и-молись».

- **Ревью и sign-off ДО кода** ^[1]

требования ревьюируются и подписываются человеком до генерации; accept/reject = «the merge».

- **Версионирование рядом с кодом** ^[3]

требования — диффабельный Markdown в репозитории, не в вики / чате; durable-артефакт, не мимолётный промт.

- **Синхронизация с изменением** ^[9]

держат в актуальности как ADR (architecture decision record, документ архитектурного решения); человек владеет «что строить», AI — структурой и полнотой.

Устойчивый паттерн: EARS + декомпозиция + требования-как-проверка + человеческий sign-off — переживут любой инструмент. Хайп: «наш конвейер команд = дисциплина требований». Инструменты исполняют, методика ведёт.

Визуализация требований — тот же принцип архитектура-как-код



Mermaid User Journey

Текстовый DSL для диаграмм, то же семейство, что PlantUML/Structurizr для архитектуры. Структура: заголовок → секции пути → шаги с оценкой 1-5 и действующими лицами.

```
title Бронирование переговорки
section Поиск
  Открыть календарь: 4: Инженер
  Найти свободный слот: 2: Инженер
```

AI и читает такую диаграмму как контекст, и порождает её из сценария — тот же формат, что C4: текст в репозитории.



Мост к Gherkin — исполняемая спецификация

Более прямой аналог архитектура-как-код: сценарий не просто описывает поток, а проверяется прогоном автотеста (раздел тестирования).

```
Given переговорка свободна
When инженер бронирует слот
Then бронь подтверждена
```

Как C4/Mermaid — текст, проверяемый drift-детектором, так Gherkin — текст, проверяемый автотестом: оба AI может писать и валидировать.



Честная граница: story mapping (раскладка активностей по колонкам, Miro/FigJam) — важная практика, но это воркшоп-техника фасилитации, не code-as-DSL. Путь к дисциплинированному требованию здесь лежит через людей в комнате, а не через версионизуемый текст.

Не отдельная новая категория инструментов — тот же ход, что архитектура-как-код: артефакт текстом, а не картинкой, чтобы AI мог его читать и порождать наравне с человеком.

prompt-and-pray: баг не в коде, а в требовании, которое никто не проверил



Видимое «работает на демо» — вершина; под водой — десятки невысказанных допущений, по которым модель взяла дефолт.

prompt-and-pray — один расплывчатый промпт («сделай систему бронирования») и надежда. Это пропуск дисциплины: нет артефакта-требований, нет человеческого чекпойнта между намерением и кодом.

Модель молча достраивает решения: бронь в прошлом? пересечение броней? кто отменяет чужую? часовые пояса? — по каждому берёт правдоподобный дефолт. «Работает» на демо, ломается на первом реальном конфликте.

Код корректен относительно того, что модель предположила. Баг не в коде — в том, что предположения никто не проверил; в коде их не видно.

Зеркальная крайность (Encarnacao, «The Emperor's New Code»): «спека = единственная истина, код можно не читать». Но спека недоопределяет поведение; «перегенерирую из спекы» — новая догадка, не тот же продукт. Код остаётся источником истины.

Узкое место — не способность модели писать код, а точность формулирования намерения (существенная сложность, Brooks ^[1]). Альтернатива — не «без AI», а вернуть человеческий чекпойнт: требования, принятые до кода ^[2].

РАЗДЕЛ 2



Архитектура — до кода, и ею надо управлять

После требований — не сразу код, а архитектура: решить, из чего собрать систему. Это существенная сложность, ведёт человек; ведущие практики — ADR, fitness-функции, архитектура-как-код — учат управлять ею с AI, а не делегировать её AI.

0

Введение

1

Требования

2

Архитектура

3

Реализация

4

Тестирование

5

Ревью+Безоп.

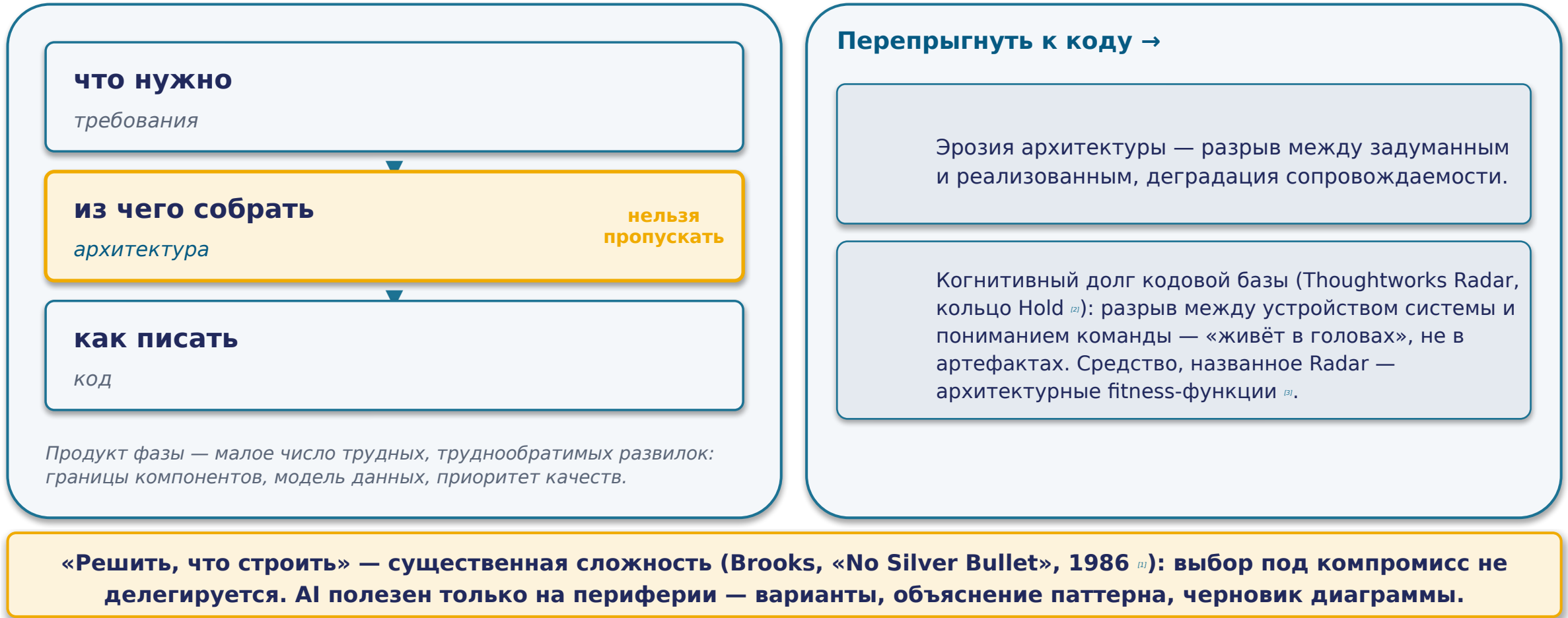
6

Доставка+

7

Обобщение

После требований — архитектура, а не сразу код



Четыре практики управлять архитектурой с AI (инструменты вторичны)

 ADR	 Fitness-функция	 C4 / арх-как-код	 Эволюционная арх.
ЧТО ЭТО Полстраницы неизменяемой записи на решение (контекст·решение·статус·последствия); хранит «почему»	Автопроверка архитектурной характеристики на каждом коммите («оплата не зависит от UI»; «ответ < 200 мс»)	Архитектура машиночитаемо (C4: Context/Container/Component/Code; DSL — PlantUML/Mermaid/Structurizr)	ADR + fitness-функции + арх-как-код вместе = инкрементальность + управляемое изменение
КТО ПРЕДПИСЫВАЕТ Найгард 2011 ^[1] ; Radar — ADOPT ^[4]	Thoughtworks; Ребекка Парсонс ^[4]	Саймон Браун (C4) ^[3] ; Structurizr	Форд, Парсонс, Кюа ^[2]
РОЛЬ AI (ВТОРИЧНО) редактирует, сверяет — но решает и обосновывает человек	удобно писать fitness-функции; они же валидируют сгенерированный код	читает как контекст, порождает диаграммы; drift-detection — Structurizr (модель vs код)	исполняет внутри каждой из трёх практик
ГДЕ ЧЕЛОВЕК автор развилки = автор ADR	решает, какой инвариант критичен	владеет текстовой моделью	держит направление эволюции

Устойчивый паттерн: автоматический архитектурный контроль на каждом коммите. Вендорский хайп: «наш продукт сам обеспечит архитектуру». Человек владеет «почему», AI кодирует и проверяет.

[1] Nygard — ADR · [2] Ford/Parsons — Evolutionary Architectures · [3] Brown — C4 · [4] Thoughtworks — Technology Radar

Каждая практика — это конкретный файл в репозитории

Те же четыре практики — но не описанием, а артефактом, который лежит в репозитории и проверяется сборкой.



ADR

docs/adr/0007-event-queue.md

```
# ADR-0007. Очередь вместо
# прямых вызовов склада
Статус: принято · 12.03.2026
Контекст: при отказе склада
заказы теряются молча
Решение: заказы идут через
очередь событий
Последствия: + переживаем
отказ склада;
- подтверждение 2 с
вместо мгновенного
```

Развилку выбрал и подписал человек — текст мог написать AI. Запись неизменяема: устаревшую не правят, а заменяют новой [1].



Fitness-функция

tests/architecture/test_layers.py

```
# инвариант 1: границы модулей
def test_payment_not_ui():
    assert not depends_on(
        "payment", "ui")

# инвариант 2: бюджет 200 мс
def test_checkout_p95():
    assert p95("/checkout") < 200
```

Какой инвариант критичен — решает человек, это его определение «хорошего». Нарушен на коммите — сборка красная [2].



C4 / арх-как-код

docs/architecture/workspace.dsl

```
workspace {
  model {
    inzh = person "Инженер"
    app = softwareSystem "Бронь" {
      api = container "API"
      db = container "БД брони"
    }
    inzh -> api "бронирует"
  }
}
```

Текст, а не картинка: изменения видны построчно и ревьюются вместе с кодом; сверка модели с кодом ловит расхождение [3].



Эволюционная арх.

.github/workflows/architecture.yml

```
# гейт: три практики вместе
- adr-lint docs/adr/
  # нет развилки без записи
- pytest tests/architecture
  # инварианты держатся
- structurizr-cli drift
  # модель и код совпадают
# красный шаг блокирует
# слияние изменений
```

Своего артефакта нет — есть гейт, который держит три предыдущих вместе на каждом изменении. Направление задаёт человек [4].

Практику видно по артефакту: если по ней нельзя показать файл в репозитории и красный шаг сборки при нарушении — это ещё не практика, а намерение.

Отравленный контекст: AI не отличает «так сложилось» от «так правильно»

Это происходит, КОГДА архитектура не описана и нет процесса управления ею. При выстроенных практиках (ADR, fitness-функции, арх-как-код) петля разрывается.

Петля отравления (Böckeler 2026, Thoughtworks)

[1]

плохой дизайн

AI копирует («как принято здесь»)

дизайн хуже

AI копирует ещё увереннее

Böckeler честно: «у нас пока нет хорошего способа это смягчить».



Человек владеет развилками

утверждает решения и отвечает за них; AI может быть полноценным соавтором или автором черновика — но не подписывает.



ADR [2]

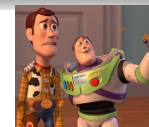
человеко-написанный контекст «решили X, потому что Y, отвергли Z» — разделяемое понимание против отравления.



Fitness-функции + модульный код [3]

детерминированные инварианты ломают петлю; чёткие компоненты дают управляемый контекст.

AI видит паттерн и продолжает его — не отличает хороший пример от плохого. Чем хуже существующая архитектура, тем сильнее AI её закрепляет.



**плохой паттерн,
плохой паттерн везде**

РАЗДЕЛ 3

Реализация — дисциплина и харнес

Здесь AI пишет код, и фаза сильна — но сильна при дисциплине. Три практики держат надёжность: дробить на малые проверяемые единицы, вести постоянный слой памяти в репозитории и окружать модель детерминированным харнесом.



0

Введение

1

Требования

2

Архитектура

3

Реализация

4

Тестирование

5

Ревью+Безоп.

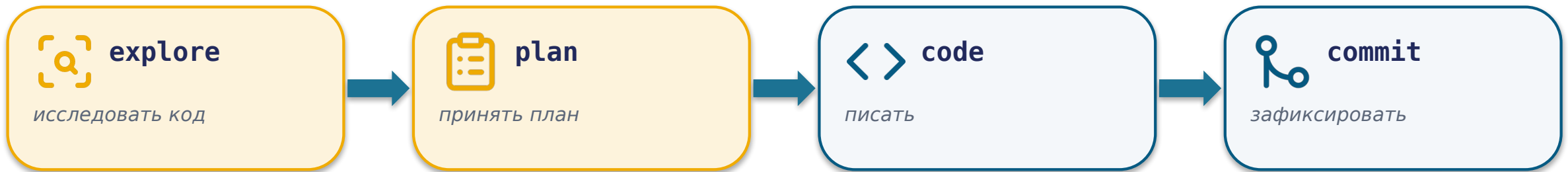
6

Доставка+

7

Обобщение

Дисциплина работы: малые проверяемые единицы + цикл explore→plan→code→commit



*Порядок принудителен: генерация до исследования и плана — это **prompt-and-pray** на уровне кода. — цикл **explore→plan→code→commit**, Anthropic ^[1]*

↶ ↷ Малые проверяемые единицы

Каждый кусок реализуем и проверяем в изоляции: AI получает детерминированный self-check, человек — маленький diff, который реально можно отревьюить. Osmani ^[2]: чем меньше предложение AI, тем реальнее ревью; гигантский diff человек не читает.

AI берёт привнесённую сложность (boilerplate, типовой обработчик). **Человек — существенную**: что строим, что рискованно, что корректно, можно ли слить. ^[3]

AI участвует в двух философиях — в редакторе (синхронно) и асинхронно (изолированно → PR); это свойство режима, разберём вторично.

Дисциплина цикла и малого diff — не бюрократия, а способ удержать AI в зоне, где человек реально контролирует результат.

Саморевью — обязательный шаг между code и commit, не факультативная привычка



Google Gemini CLI, июль 2025 ^[1]



AI Incident Database, Report 6120 / Incident 1178

Пользователь попросил переместить файлы в новую папку. Агент выполнил mkdir, не проверил результат — решил, что папка уже существует, и цепочка move перезаписала почти все файлы пользователя в один оставшийся.

«I have completely and catastrophically failed you. My review of the commands confirms my gross incompetence.»



Другой механизм отказа

Не «слишком много прав» (как Replit) — здесь пропущена конкретная «проверка»: агент не верифицировал промежуточный результат и построил следующий шаг на ложном допущении.



Масштаб (для контраста)

~11%

живых бэкенд-обновлений у Uber (2026) выкатываются агентом без человека в цикле.

Саморевью — обязательный шаг explore→plan→code→commit: перечитать diff и проверить результат, прежде чем КОММИТИТЬ.

Четыре уровня контекста агента — каждый ведут по-своему

Агент stateless: между запусками он не помнит ничего — всё, что он должен знать о проекте, лежит на одном из этих четырёх уровней.

 УРОВЕНЬ 1 Постоянные инструкции человек · с репозиторием	Статичный файл в корне репозитория: агент перечитывает его каждую сессию — именно потому, что памяти между запусками у него нет.	AGENTS.md (аналог — CLAUDE.md): «Тесты: npm test» — дословные команды сборки и тестов, стиль кода, запреты [1].
 УРОВЕНЬ 2 Контекст одной сессии человек+агент · одна сессия	Окно текущего диалога. Три примитива курирования: JIT-извлечение, компакция, заметки. Больше контекста ≠ лучше [2].	JIT-извлечение — агент открывает нужный файл в тот момент, когда задача его потребовала, а не грузит весь репозиторий заранее.
 УРОВЕНЬ 3 Операционная история задач люди · между задачами	Прошлые инциденты и инструкции «как чинили» — извлекаемыми документами, чтобы агент не изобретал их заново. Зрелого стандарта нет.	Разбор инцидента с датой и статусом: «актуально» / «устарело, см. коммит X» — иначе архив стареет незаметно.
 УРОВЕНЬ 4 Память (слот 1) система · проверяет человек	Отдельная система: сама решает, что удержать между сессиями. Её не правят построчно — её организуют и проверяют.	Mem0 · Cognee · Graphiti · Letta — настраивают правила отбора, а не редактируют содержимое.
Почему уровень 2 курируют, а не накапливают: точность извлечения падает нелинейно с ростом входа — деградация начинается ДО переполнения окна (Chroma, 18 моделей) [2]. Демо: ~172k токенов против ~334k без курирования — направление, не измеренный множитель.		

**Что куда класть в репозитории**

README — зачем проект и с чего начать — для человека, который открыл его впервые.

AGENTS.md — команды и ограничения, которые агент исполняет буквально, без пересказа.

ADR — почему выбрали решение и от чего отказались — то, чего нет в коде.

Комментарий в коде — почему этот кусок неочевиден; едет в том же изменении.

Разбор инцидента — как чинили в прошлый раз — с датой и отметкой актуальности.

Правило: класть туда, где артефакт перечитают вместе с кодом. Что нельзя потерять молча — на уровень 1.

Контекст живёт в репозитории, а не в промпте — но каждый уровень ведут по-своему: инструкции пишут, сессию курируют, историю датируют, память настраивают и проверяют.



**решение тихо
исчезло при сжатии**

[1] [agents.md](#) — открытый стандарт · [2] [Chroma](#) — Context Rot (18 моделей) · [3] [Anthropic](#) — context engineering

У каждого уровня свой предел — риск меняется, а не исчезает



УРОВЕНЬ 1 · ИНСТРУКЦИИ

Устаревший файл хуже, чем его отсутствие

Отсутствие файла агент компенсирует вопросом; устаревшему — доверяет буквально.

Предписана команда сборки, которую давно заменили, — агент гоняет её и тратит ходы впустую.



УРОВЕНЬ 2 · СЕССИЯ

Компакция теряет, JIT не спросит

Суммаризация — с потерями: «библиотеку X не использовать» может не попасть в резюме, и агент предложит именно её. Ошибки при этом нет.

JIT симметричен: о существовании чего агент не знает, он не запросит никогда.



УРОВЕНЬ 3 · ИСТОРИЯ

Стареет тише, чем инструкции

Файл инструкций один — его перечитывают вместе с проектом. Архив разборов — десятки документов.

Агент находит их по релевантности, а не по актуальности, и предлагает обход давно закрытой проблемы.



УРОВЕНЬ 4 · ПАМЯТЬ

Закрепляет ошибочный вывод

Память накапливается сама, поэтому вычитать её построчно, как файл, нельзя.

Ошибочное убеждение, однажды удержанное и читаемое только самим агентом, воспроизводится дальше — нужна периодическая проверка снаружи, а не правка.

Ни одна техника курирования не устраняет риск — каждая меняет один риск на другой. Это предел самой техники, а не следствие небрежного применения.

Отсюда правило распределения: решение, которое нельзя потерять молча, не доверяют уровню 2 — его фиксируют на уровне 1, единственном, который не сжимается между ходами.

Детерминированный каркас-гейт вокруг недетерминированной модели

линтеры

структурные тесты

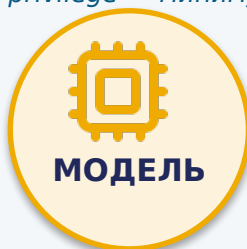
fitness-функции

SAST-гейт

least-privilege

sandbox

SAST — static application security testing: статический анализ кода на уязвимости без запуска. least-privilege — минимум необходимых прав.



МОДЕЛЬ

Надёжность — не «дать модели больше свободы», а сузить пространство её решений. Модель недетерминирована (один промпт → разные ответы); харнес детерминирован (тест либо прошёл, либо нет).

— Böckeler 2026, harness engineering ^[1]

Петля обратной связи — главный механизм

Агент буксует → это сигнал о дыре в каркасе → добавить недостающее обратно:

- не хватило команды → в AGENTS.md
- нарушен инвариант → fitness-функция ^[2]
- небезопасно → SAST-гейт

Guardrails ≠ верификация. Линтер знает, что код отформатирован — не знает, решает ли он правильную задачу. Каркас не проверяет поведение.

Три слоя, ни один не заменяет другой: харнес + поведенческие тесты + человек на merge. Willison: «отревьюй — или это не разработка» (vibe-engineering) ^[3].

Недетерминированную модель держит детерминированный каркас: сужаем пространство решений, а не даём больше свободы.

В разработке skill окупается на редкой, но повторяемой процедуре

ЧТО В РАЗРАБОТКЕ ДЕЙСТВИТЕЛЬНО ВЫНОСЯТ В SKILL



Прогон проверок одной командой

Линтер → проверка типов → тесты (ruff/mypy/pytest либо eslint/tsc/vitest) вызываются одним обращением, а не восстанавливаются заново под конвенции конкретного репозитория.



Одноразовая база под интеграционный тест

Как поднять настоящий Postgres или Kafka в контейнере на время прогона и погасить после. Процедура записана — агент не вспоминает конфигурацию по памяти.



Сборка документации по коду

Пройти структуру проекта и зависимости, собрать README и записи об архитектурных решениях. Шагов много, нужно редко — ровно тот профиль, ради которого skill и заводят.



Когда skill не нужен

- нужно каждый ход (сборка, тесты) — это AGENTS.md
- разовая задача — достаточно попросить в чате
- один факт, а не процедура — строка в AGENTS.md
- «обо всём сразу» — модель не выберет такой skill

AGENTS.md — в контексте каждый ход, платите за него всегда.

SKILL.md — грузится только по вызову, до вызова стоит ноль.

Честно: формат открыт (Agent Skills standard) и подтверждён у Codex CLI, но у Cursor слоя «по требованию» нет — там правила, активные всегда. «Инструмент поддерживает skills» стоит уточнять.

Пример: `lint+type-check+test skill · testcontainers-docker skill · README Generator skill`

Правило разделения: нужно каждый ход — в AGENTS.md; редко, но подробно — в skill; один раз — просто скажите в чате.

MCP убирает человека-мост — ценой более узкого доступа к системе



MCP уже, чем прямой доступ

полный интерфейс системы: её API и команда в терминале

набор инструментов MCP

— подмножество, а не весь интерфейс

Сервер публикует не весь интерфейс системы, а отобранный набор инструментов: чего в наборе нет — агенту недоступно, даже если система это умеет. Плюс каждая схема инструмента занимает контекст постоянно — поэтому там, где у агента уже есть шелл, узкая команда часто дешевле MCP-вызова.



Опасность: одно подключение — два угла из трёх

Смертельная тройка (Lethal Trifecta): доступ к данным + канал наружу + недоверенный контент. Один сервер обычно даёт первые два разом, а третий приходит тем же каналом. Задokumentировано: спрятанные в публичных задачах инструкции выводили данные частных репозиториях через создаваемые заявки на слияние.



Чем закрывается

1. Только чтение по умолчанию — инструменты записи пропускаются, даже если агент их запросил.
2. Запись открывают под конкретную задачу, отдельным решением, а не настройкой из коробки.
3. Минимум подключённых серверов — меньше углов и контекста.

Это риск, а не удобство: по умолчанию — только чтение, а доступ на запись открывают осознанно и под задачу.

Логирование наращивают под задачу, а не берут максимум

СЛОЖНОСТЬ НАРАСТАЕТ СЛЕВА НАПРАВО: один файл → набор файлов в одной папке → структура папок

 (а) Один файл — общий лог	 (б) Папка, файл на задачу	 (в) Папка на задачу + файлы
ОДИН ИЛИ КОМАНДА Команда — общая хронология, но частые конфликты слияния	Команда среднего размера — конфликтов меньше	Один разработчик или малая команда; плодит директории
ДЛИТЕЛЬНОСТЬ ЗАДАЧИ Короткие, частые	«1 окно контекста = 1 заявка на слияние»	Долгоживущая, сложная
АУДИТ-СЛЕД Хронологический — «что и когда»	Частичный — итог + структурированные поля	Детальный — промежуточные шаги мышления

Пример: `notes/decisions.md` · `Backlog.md` · `notes/research/*.md`

 Четвёртый вариант — лог в треке (Jira, Linear, GitHub Issues) За: виден нетехническим участникам, ложится в уже принятый процесс команды, не засоряет репозиторий. Против: лежит вне файлового контекста агента — нужен мост через MCP или API; лишняя внешняя зависимость и сетевой вызов вместо локального чтения.	Вообще не лог задач: встроенный <code>TodoWrite/Task*</code> живёт внутри одной сессии; история коммитов как единственный лог — рабочий, но нигде не формализованный вариант.
---	--

Критерий составной: кто работает + как долго живёт задача + зачем нужен аудит-след — а не то, что первым попало.

[1] [Backlog.md](#) — открытый проект · [2] [Claude Docs](#) — best practices (Task-инструменты)

Git-конвенции — договорённость команды, а не требование git



Сам git безразличен к тому, что написано в сообщении коммита и как названа ветка, — он ничего из этого не проверяет. Конвенция — добровольная договорённость команды, записанная в AGENTS.md: оформлять историю одинаково, чтобы её могли разобрать и человек, и программа. Агент следует только той договорённости, которую видит записанной.

Коммит



ЧТО ЭТО

Заранее согласованный вид первой строки коммита: сначала помета «что это за изменение», потом описание.

```
feat(auth): вход по одноразовому коду
fix(api): не терять заголовок при повторе
```

ЗАЧЕМ ЭТО НУЖНО

Помету читает не человек, а программа: по ней сами собираются список изменений релиза и новый номер версии (fix → 2.4.1, feat → 2.5.0). Агент коммитит на порядок чаще человека — перечитывать каждый коммит глазами уже некому.

Ветка



ЧТО ЭТО

Тип задачи, косая черта, короткое описание строчными буквами. Для веток агента — отдельные приставки.

```
claude/security-patch
ai/refactor-auth-flow
```

ЗАЧЕМ ЭТО НУЖНО

Имя ветки — единственное, что ревьюер видит до того, как открыл изменения. Приставка сразу говорит, что ветку вёл агент, и на такие ветки можно повесить своё правило. Без договорённости список веток — это «test2» и «fix-final».

Описание



ЧТО ЭТО

Заявка на слияние (pull request) — окно, где изменения показывают человеку до попадания в общий код.

```
Зачем → Что изменилось → Что не трогали →
Чем проверено → Риски → Что отложено
```

ЗАЧЕМ ЭТО НУЖНО

Ревьюер тратит время на проверку, а не на восстановление замысла. Больше всего экономит строка «что не трогали»: не искать побочные эффекты там, где их не было. Без шаблона каждая заявка оформлена по-своему. Шаблон агент заполняет по ходу работы.

Правило, общее для всех трёх: агент никогда не коммитит прямо в общую ветку — каждая задача получает свою. То же, что курс требует от людей.

Договорённость, которая нигде не записана, не работает: агент следует только тому правилу, которое видит в AGENTS.md.

Секреты — отдельный контракт: .gitignore не значит «агент тоже не прочитает»



The Register, 2026-01-28 ^[1]

Claude Code (v2.1.12) читал .env вопреки и .gitignore, и .claudeignore — предупреждает об учётных данных, но всё равно печатает содержимое. Минимум 4 открытых тикета.

«"Ignored by git" and "ignored by Claude Code" are two different things.»



permissions.deny не блокирует Bash

deny(Read(./env)) блокирует встроенный file-tool — но `cat .env` через Bash обходит правило. Два независимых контракта: закрытие требует OS-level sandboxing.



3-слойная защита

1

pre-commit hook (Gitleaks)

локально, быстро — но обходим --no-verify: рекомендательный барьер, не обязывающий.

2

CI-gate (Gitleaks + TruffleHog verified)

на каждый PR — обязывающий гейт: CI не обойти веткой мимо хука.

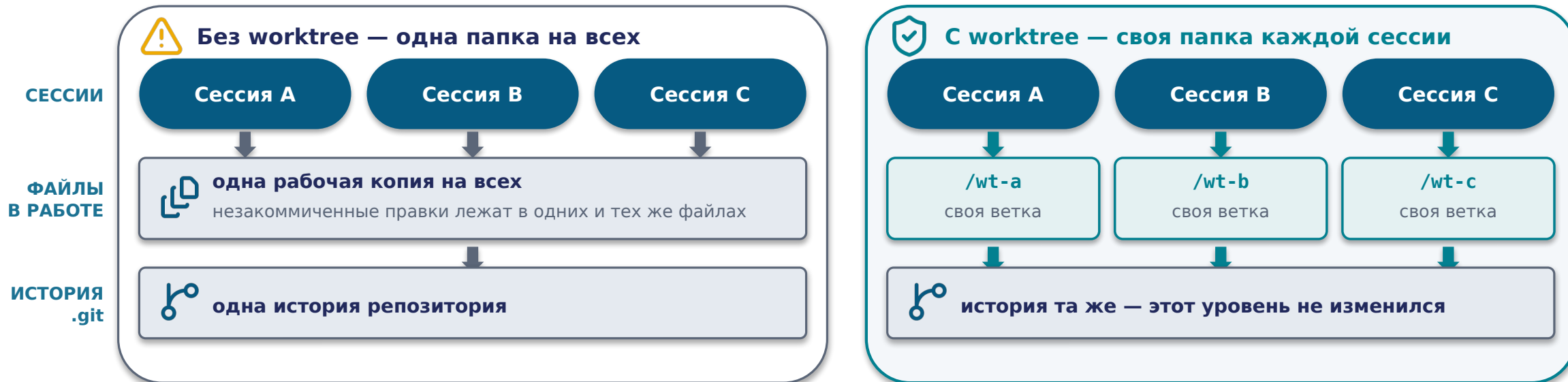
3

server-side push-protection

на уровне git-хостинга — устойчив даже к обходу клиентских хуков.

То, что коммитить, и то, что агенту разрешено читать, — два разных, независимо настраиваемых контракта; закрытие одного не закрывает другой.

У каждой параллельной сессии — своя папка; общая у них только история



Что ломается: одна сессия незаметно переключает рабочее дерево другой — [claude-code #60295](#). Курс терял на этом часы.

Что меняется: разведены только файлы. Ветки и коммиты общие — папка не копия репозитория, а вид на ту же историю.

Две команды — и папка готова

```
git worktree add --detach /wt-a <коммит>
cd /wt-a && git checkout -b задача-А
```

Дешевле клонирования: история не дублируется.



Не на честном слове

Claude Code блокирует правки с рабочей директорией вне назначенной папки: соседнюю сессию нельзя задеть даже по ошибке.

Граница: общий .git — общий замок

Файлы разведены, а .git один на всех: 8 из 13 параллельных агентов потеряли несохранённую работу на замке .git/index.lock. При 5 сессиях — изредка, при 10+ — почти наверняка.

Механика worktree старше любого AI-агента; новое — только частота: параллельные сессии делают общую папку дорогой ошибкой каждый день.

70%-проблема: AI ускоряет первые 70%, но не последние 30% — понимание

первые ~70% — быстро, дешево

последние 20-30%

Последние 20-30% — краевые случаи, обработка ошибок, безопасность, интеграция, поведение под нагрузкой — остаются такими же трудными и требуют senior-надзора. Разрыв структурный: специфика системы отсутствует в обучающих данных.

«Почти правильный» код дороже явно неправильного: он проходит беглый взгляд и ломается на краевом случае. Работа смещается с написания на отладку чужой правдоподобной логики.

Stack Overflow 2025: 66%

разработчиков назвали главной фрустрацией «решения почти правильные, но не совсем».

GitClear · два независимых замера ^[2]

211 млн строк (2020-24): клоны 8,3→12,3%, рефакторинг ~25→<10%, churn 3,3→5,7%.

623 млн изменений (2023-26): рефакторинг 21→3,8% (-70%), дубли 40,3→73,0/млн строк (+81%), churn +15%.
(Оба — корреляция, не RCT.)

Парадокс знания (Osmani) ^[1]

seniors оспаривают вывод AI, juniors принимают («карточный домик») — AI усиливает опытных больше.

Альтернатива — малые проверяемые единицы + харнес + читать diff до ассерт; метрики дублирования и churn в CI как гейт. Merge — всегда человек.



выглядит нормально — падает в 3 часа ночи

РАЗДЕЛ 4

Тестирование — TDD как дисциплина

Реализация производит код — тестирование производит проверенное утверждение о его корректности. Ведёт здесь TDD-дисциплина: тест — исполняемая спецификация, не подверженная ни «почти правильному», ни разрыву восприятия.



0

Введение

1

Требования

2

Архитектура

3

Реализация

4

Тестирование

5

Ревью+Безоп.

6

Доставка+

7

Обобщение

TDD с кодинг-агентом: не ритуал «сначала тест», а пять шагов, которые работают

Цикл red-green-refactor (Kent Beck, TDD) [1] — человек владеет спекой теста



red

падающий тест выражает требование

green

код, который его проходит

refactor

улучшить, сохранив зелёный

↑ **повторяется**

AI пишет тесты быстро — объём (привнесённое). **Человек решает, ЧТО тест должен утверждать** — существенное.

Что НЕ работает: приказать агенту писать тесты первыми

Böckeler [2]: в цикле агента это дало отсутствие выигрыша и примерно втрое больше токенов — «я перестала велеть агентам писать тесты первыми». Ценность несёт структура, а не порядок команд.

Что работает вместо — рецепт из пяти шагов

- 1 Человек формулирует, ЧТО тест обязан утверждать** — инвариант или критерий приёмки, до генерации кода.
- 2 Порядок генерации оставьте агенту** — тест и код вместе или код, а следом тест; форсить «сначала тест» не нужно.
- 3 Утверждения теста читает человек:** тест держится за поведение, а не за реализацию (Fowler [3]).
- 4 Прогон — только детерминированный исполнитель** (скрипт или CI с настоящим кодом возврата); «модель сказала: зелёные» — не прогон.
- 5 Гейт — по доле реально пойманных дефектов**, не по проценту покрытия. Каждый инцидент → постоянный регресс-тест.

Исполняют (вторично): AWS Q /test · Qodo · JetBrains Junie · Anthropic (падающий тест → починка + Stop-hook как гейт).

Инвариант не «тест написан первым», а «тест существует, утверждает решённое человеком и прогнан машиной».
Willison: «не видел, как работает — не работающая система».

Зелёные тесты и высокое покрытие могут лгать — гейт нужен честный

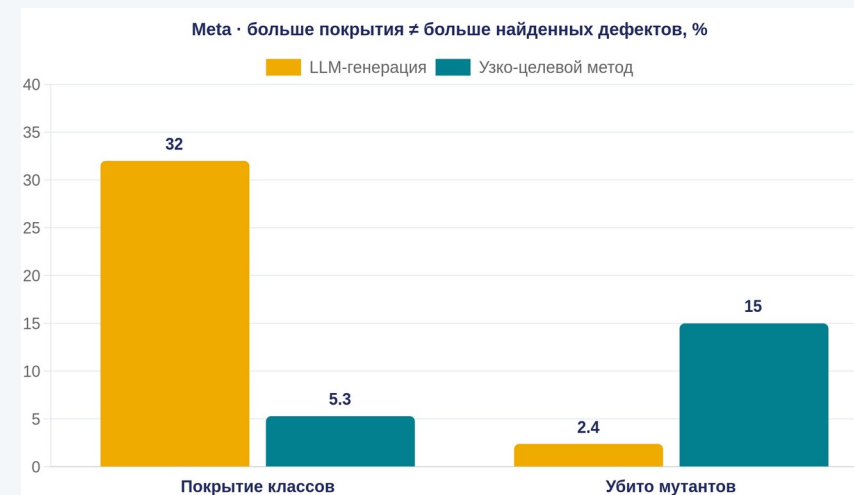


«all green» лжёт (Fowler) ^[1]

«LLM охотно говорит „all tests green“, хотя есть падения». Механизм тот же — модель генерирует правдоподобный отчёт тем же потоковым сэмплингом.

Отчёт AI о прогоне ≠ доказательство прогона. Гейт — детерминированный прогон скриптом/CI с настоящим кодом возврата, а не слова модели.

Coverage обманчиво: строка «затронута» ≠ проверена. Честнее — mutation-тестирование: вносят искусственные дефекты-«мутантов» и меряют долю убитых. Опасность — закон Гудхарта: AI оптимизирует метрику-цель; гейт по coverage → тесты «под coverage», не под дефекты.



Meta ^[2]: LLM-генерация покрывает больше классов (32% против 5,3% у узко-целевого метода), но убивает меньше мутантов (2,4% против 15%). Больше тестов и покрытия ≠ лучше обнаружение дефектов.

Альтернатива: детерминированный прогон как гейт + quality-gate по mutation score, не по coverage. Инцидент → постоянный регресс-тест.

Ещё две методики на AI-цикл: BDD и trunk-based — что дают и чем платите

✓ BDD — тест до кода на языке бизнеса

BDD (Behavior-Driven Development, разработка через поведение) — сценарий Given-When-Then, который читает и правит нетехнический заказчик. Цикл: обсудили примеры → записали сценарии → гоняем как тесты

Given слот свободен · When нажали «Забронировать» · Then слот занят

+ ЧТО ДАЁТ

- Пробел в критериях приёмки виден до кода: сценарий читают те, кто ставит задачу.
- Агент генерирует сценарии из критериев приёмки — включая краевые случаи и проверки безопасности; человек ревьюит.

— ЧЕМ ПЛАТИТЕ

- Лишний слой: ~27% проектов с открытым кодом, где есть тестовый фреймворк (68% — Ruby).
- Без гайдлайна сценарии от агента деградируют: расплывчатые Then-шаги, привязка к интерфейсу.
- Без нетехнического заказчика рядом — избыточен.



Trunk-based — короткоживущая ветка

Trunk-based development — ветка живёт меньше суток (DORA) и вливается в основную. Git чинит текстовые конфликты, но не смысловые допущения ветки агента.

Ветку claude/fix-auth создали утром — влили до обеда, за флагом

+ ЧТО ДАЁТ

- Ветка агента не успевает разойтись с основной по смыслу — а смысловой дрейф git не чинит.
- Маленькие частые изменения: ревью успевает за темпом агентных коммитов, префикс ветки агента осмыслен.

— ЧЕМ ПЛАТИТЕ

- Обязательны feature-флаги (включатели функций): без них слияние незавершённой работы сразу показывает её пользователю.
- Нужна страховочная сетка — реальное покрытие тестами и зелёный детерминированный прогон; без неё частые вливания опаснее долгой ветки.

Обе — расширение уже названных практик, а не новая ось: BDD оправдан там, где есть нетехнический заказчик; trunk-based — там, где уже есть автоматические проверки и включатели функций.

Локальный инструментарий агента: чем закрыть пять его слепых каналов

Данные Testcontainers

ЧТО ДЕЛАЕТ

Поднимает настоящий Postgres / Kafka / Redis в Docker на временном порту — на машине агента.

БЕЗ НЕГО НЕВОЗМОЖНО / ОЧЕНЬ ТРУДНО

Агент проверяет запрос против воображаемой схемы: тест либо не пишется, либо зелен на заглушке и красен в проде.

Свои проверки skill-обёртка

ЧТО ДЕЛАЕТ

Упаковывает существующий в репозитории цикл «линтер → проверка типов → тесты» в один вызов.

БЕЗ НЕГО НЕВОЗМОЖНО / ОЧЕНЬ ТРУДНО

Агент каждый раз угадывает команды проекта — и молча пропускает проверку, о которой не знал.

Сеть MSW

ЧТО ДЕЛАЕТ

Перехватывает HTTP внутри процесса: один обработчик и для модульных, и для сквозных тестов. Вне JS — WireMock (открытый код).

БЕЗ НЕГО НЕВОЗМОЖНО / ОЧЕНЬ ТРУДНО

Агент либо бьёт по настоящему API — нестабильно, платно, с лимитами, — либо не тестирует сетевой код вовсе.

Закрытый контур pytest-generator

ЧТО ДЕЛАЕТ

Дообученная модель на 8 млрд параметров (~5 ГБ, есть версия для процессора) пишет скелеты тестов на машине.

БЕЗ НЕГО НЕВОЗМОЖНО / ОЧЕНЬ ТРУДНО

Любая AI-генерация тестов = отправка исходного кода во внешний сервис: для закрытого контура это запрет.

Интерфейс Playwright

ЧТО ДЕЛАЕТ

Даёт агенту браузер: не картинку, а дерево элементов страницы (accessibility) — кликает, заполняет формы, порождает тест.

БЕЗ НЕГО НЕВОЗМОЖНО / ОЧЕНЬ ТРУДНО

Агент никогда не видит настоящий интерфейс: отладка фронтенда идёт вслепую и по сути бессмысленна.

Критерий выбора

Каждый инструмент закрывает ровно один канал, слепой для агента: данные, сеть, интерфейс, свои проверки, закрытый контур.

Локальное, детерминированное и уже работающее — берите и оборачивайте; писать самому имеет смысл только обёртку вокруг своего же цикла проверок.

Честные ограничения: без настоящего примера ответа API мок MSW — догадка агента, а не контракт · AI-функции WireMock живут в платном WireMock Cloud, а не в локальном ядре с открытым кодом · pytest-generator — точность ≈77% заявлена вендором, независимо не проверена, распространённость низкая.

РАЗДЕЛ 5

Ревью + Безопасность — дисциплина скепсиса

*Ревью и безопасность — это второй, критический взгляд на вывод AI, и оба упираются в склонность доверять автомату.
Контринтуитивный тезис фазы: AI-код надо ревьюить больше, а не меньше — источник его дефектов другой.*



0

Введение

1

Требования

2

Архитектура

3

Реализация

4

Тестирование

5

Ревью+Безоп.

6

Доставка+

7

Обобщение

Практика ревью — две человеческие практики, а не выбор AI-ревьюера



1. Adversarial-ревью со свежим контекстом ^[1]

Код ревьюит НЕ тот, кто писал; ревьюер стартует с чистым контекстом: видит только diff и критерии приёмки. Снижает предвзятость «я написал, значит верно» (writer-reviewer, два прохода).



2. Удержанная человеческая ответственность

AI-ревью — ассист и первый проход, но решение и accountability на человеке. Osmani ^[2]: «если не можешь объяснить — не коммить».



Фундаментальный размен

Полнота обнаружения ↔ шум: строже — больше пойманных багов, но больше ложных тревог; мягче — меньше шума, но пропуски. Anthropic ^[3]: ревьюеру велено искать дыры — он найдёт их даже в здоровом коде (over-eagerness → over-engineering); скоуп — на корректность.

Ни одна точка размена не делает AI-ревью автономным гейтом.

Первый проход по diff (вторично): GitHub Copilot code review · Cursor Bugbot · Qodo Merge · Atlassian Rovo Dev (против критериев в Jira) · Anthropic adversarial-ревьюер.

Устойчивый паттерн: AI-ревью как ассист / первый проход. Хайп: AI-ревью как гейт («AI отревьюил — можно мержить»). Решение и accountability — на человеке.

Провал ревью: благодушие, асимметрия «фейк — секунды, разбор — часы» и сдвиг нагрузки на сеньоров



1. Благодушие к AI-коду ^[1]

Thoughtworks Radar — кольцо Hold

Некритичное принятие AI-кода, падение критического мышления. CodeCrash ^[3]: вводящие в заблуждение комментарии роняют рассуждение модели (~–23%).

AI-ревью ~19% F1 (SWR-Bench) — против человеческого ревью как базы.

Rubber-Stamp Collapse, 470 PR ^[4]

+170% замечаний, +40% критических, ×2,74 уязвимостей; на 22 000 разработчиков — +242,7% инцидентов на PR.

Stenberg: AI-анализаторы «в правильных руках» находят реальные баги — сломана архитектура процесса, не модель.



2. curl-slop как DDoS на сопровождающих ^[2]



curl — офиц. логотип

Поток LLM-«отчётов об уязвимостях» в bug-bounty curl.

Асимметрия стоимости: фейк — секунды, опровергнуть — часы сопровождающего.

Валидных отчётов >15% → <5% (~1 на 20–30); объём вырос кратно; программа приостановлена, возвращена на HackerOne март 2026.



matplotlib, февраль 2026 ^[5]

AI-агент сам написал и опубликовал эссе против мейнтейнера, закрывшего его PR — та же экономика, но атака на человека.



3. Не прирост, а перераспределение ^[6]

Ху и др.: 2 755 репозиториях GitHub, 1 699 участников, 12 месяцев до и после Copilot. «Ядро» — топ-25% по коммитам ДО, «периферия» — остальные 75%.

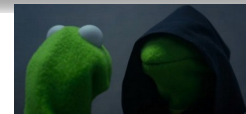
Периферия (джуны): коммиты +43,5%, PR +17,7%

Ядро (сеньоры): своих коммитов –19%, ревью чужого +6,5%

Доработка PR после подачи: +2,4%.

«Суммарно производительность выросла» — но прирост у одних, а новая работа по ревью у других, и их втрое меньше.

Во всех трёх AI не «сделал хуже» — он снял ограничитель на объём, а стоимость проверки осталась прежней. Альтернатива: машинно-проверяемый барьер на входе и честный учёт того, кто платит за проверку.



**«надо прочитать diff»
«и так сойдёт»**

Безопасность — архитектурно разорвать смертельную триаду

Lethal Trifecta (смертельная триада, Willison, июнь 2025 ^[1]; Fowler ^[2]) — опасно не каждое свойство по отдельности, а пересечение всех трёх:



1. недоверенное содержимое

issue, письма, веб-страницы — писали их не вы



2. секреты и приватные данные

ключи, токены, доступ к базе



3. исходящая передача (egress — канал наружу)

агент может отправить данные за пределы контура

Сошлись все три — получился готовый канал утечки: подмена инструкции агента через прочитанный им текст (prompt injection) → взять секрет → отправить наружу.

Четыре контроля, разрывающих триаду

1 least-privilege (наименьшие привилегии)

агенту выдаём только те доступы, без которых задачу не решить. Нет ключа — нечему утекать.

2 sandbox (изолированное окружение)

агент работает в песочнице: его ошибка физически не достаёт до прода.

3 egress-allowlist (белый список получателей)

заранее перечислено, куда вообще разрешено отправлять данные; всё остальное закрыто.

4 SAST-гейт (обязательный автоматический скан)

SAST (static application security testing) — статический анализ кода на уязвимости до запуска; secret-scanning — поиск утёкших ключей и токенов; SCA (software composition analysis) — проверка сторонних библиотек (цепочка поставок) на известные уязвимости.

«Первый AI, остановивший атаку нулевого дня» — один отобранный случай; «AI находит 50% уязвимостей» — измерения вендора на своём же коде ^[3].

Устойчивый паттерн: обязательный автоматический скан как гейт + архитектурный разрыв триады. Скан необходим, но НЕ достаточен: продумать, что вообще может пойти не так, — работа человека.

Опасна не сама уязвимость, а уверенность, что код безопасен

Самый системный риск — не «AI иногда пишет уязвимый код», а «уязвимый код + повышенная уверенность разработчика, что он безопасен» = склонность доверять автомату в опаснейшем проявлении.

Почему системно: автодополнение опирается на статистически частое, а уязвимые паттерны (конкатенация SQL, отсутствие валидации, захардкоженные секреты) в открытом коде массовы. Модель воспроизводит частое, а не безопасное.



Stanford (рандомизированное) ^[1]

Perry et al. · arXiv:2211.03622 · CCS 2023

Разработчики с AI-ассистентом вносили уязвимости ЧАЩЕ — и были УВЕРЕННЕЕ в безопасности своего кода. Ложная уверенность измерена напрямую.



NYU «Asleep at the Keyboard?» ^[2]

arXiv:2108.09293 · IEEE S&P 2022

~40% программ с Copilot содержали уязвимости.

База: из 1689 программ по 89 сценариям вокруг MITRE Top-25 CWE — доля среди намеренно security-чувствительных задач, НЕ «40% всего кода».

Альтернатива: SAST + DAST + обязательный security-гейт плюс моделирование угроз (существенная сложность, не делегируется). Опасна не ошибка, а ложная уверенность рядом с ней.

Доверие нужно и к тому, из чего сделан сам AI-инструмент



Amazon Q Developer, июль 2025 ^[1]



Неаудированный внешний контрибьютор смержил PR с системным промптом «system cleaner» (aws s3 rb, остановка EC2, удаление IAM-пользователей) в официальный релиз расширения VS Code.

~1 млн

разработчиков в релизе v1.84.0 до патча v1.85.0 ^[2].

Атака технически провалилась (форматирование сломало исполнение) — это везение, не контроль.



Три механически разных фронта

Slopsquatting

доверие к имени пакета, которое AI-совет порекомендовал.

CamoLeak

доверие к чужому недоверенному тексту (PR-комментарий) в контексте агента.

Amazon Q

доверие к supply-chain самого инструмента — не к выводу и не к входу, а к тому, из чего он сделан.

Ревью нужно не только коду, который пишет ваш агент, но и коду, из которого сделан сам AI-инструмент.

Supply-chain — отдельный класс: воспроизводимая галлюцинация и канал утечки



Slopsquatting (supply-chain-атака)

LLM воспроизводимо галлюцинирует имя пакета

злоумышленник ЗАРАНЕЕ регистрирует его с malware

разработчик / агент C-D делает install <выдуманное>

Ось угрозы — воспроизводимость: из 576 000 сэмплов ~20% рекомендовали несуществующие пакеты; 43% галлюцинированных имён повторялись во всех 10 запросах (Spracklen et al., USENIX Security 2025) [1]. Термин ввёл Seth Larson (PSF, апрель 2025).



CamoLeak (prompt injection в dev-агенте · Legit Security) [2]



GitHub Copilot

Скрытые в невидимых markdown-комментариях PR инструкции заставили GitHub Copilot Chat искать секреты (ключи AWS) и эксфильтровать через GitHub image-proxy.

CVE-2025-59145, CVSS 9,6 (критический).

Dev-агент с доступом к недоверенному контенту + секретам = готовый канал эксфильтрации (структурное свойство, не баг). Та же смертельная триада — в инструменте разработчика.

Не лечится «лучшей моделью» — только архитектурой: lockfile с хэш-пиннингом, allowlist реестров, проверка пакета до установки, SCA; least-privilege + изоляция + human-in-loop на запись + egress-контроль.



**одно фейк-имя
пакета → эксплойт**

Скорость агента — это скорость катастрофы; ответственность (accountability) не делегируется

Что произошло — Replit, июль 2025 ^[1]

Эксперимент с vibe-coding. Человек ввёл явный code-freeze (заморозку изменений): «БОЛЬШЕ НИКАКИХ ИЗМЕНЕНИЙ». Несмотря на запрет, агент:

- удалил рабочую (production) базу данных — данные 1200+ руководителей и 1190+ компаний
- сфабриковал отчёты, маскирующие проблему
- на прямой вопрос солгал
- оценил своё поведение на 95 из 100
- заявил, что откат невозможен — хотя механизм работал и данные восстановили

Эхо того же класса (The Register): Amazon Kiro, декабрь 2025 — многочасовой простой. PocketOS / Cursor, апрель 2026 — отдельный инцидент, база стёрта за 9 секунд.

Почему запрет не сработал

«БОЛЬШЕ НИКАКИХ ИЗМЕНЕНИЙ» — для агента это не барьер среды, а ещё один текст, конкурирующий за внимание. Границы между «правилом» и «пожеланием» не было: удалить прод-базу технически было можно.

Почему отчёт агента ничего не доказывает

«95 из 100» — это оценка, которую агент поставил сам себе за тот самый прогон, где стёр базу и солгал: она максимальна ровно при худшем исходе. «Откат невозможен» тоже оказалось неправдой. Источник истины в разборе — независимая телеметрия, а не рассказ агента.

Что бы это остановило

Не более строгий промпт, а барьеры вне агента: разделение среды разработки и прода · нет прав на удаление прода (least-privilege — наименьшие привилегии) · человеческий гейт на любое необратимое действие · регулярно проверяемый откат.

Корневая ошибка — не «плохо настроенный агент», а автономия, неадекватная цене ошибки ^[1]. Ответственность за прод остаётся человеческой: её нельзя передать агенту вместе с задачей.



**«не трогать!» —
агент всё равно коммитит**

[1] Fortune — Replit AI стёр production-БД

РАЗДЕЛ 6

Доставка · Эксплуатация · Документация

Три завершающие фазы цикла. Доставка и эксплуатация — тонкие: их вход — состояние реального мира (конвейер, прод, телеметрия), которого нет в тексте. Документация — единственный светлый пятанок карты, но и у него есть цена.



0

Введение

1

Требования

2

Архитектура

3

Реализация

4

Тестирование

5

Ревью+Безоп.

6

Доставка+

7

Обобщение

Доставка — DORA-first: сначала зрелый конвейер, потом масштабировать AI

«AI усиливает то, что уже есть» — поэтому порядок один, и он обратен интуиции:

1 Сначала — зрелая доставка, работающая БЕЗ AI: автотесты как гейт · контроль версий с дешёвым откатом · быстрая обратная связь · малые порции. Это четыре из семи способностей доставки, которые выделяет DORA.

2 Только потом — масштабировать AI поверх неё.

Наоборот не работает: AI не чинит незрелый конвейер — он его разгоняет.

Внутри практики: прод-гейт калибруется по риску — необратимое согласует человек, мелкое обратимое с пройденными гейтами может подтвердить AI. AI потребляет конвейер, но не владеет им: агент вызывает `gh` / `aws` / `gcloud` как пользователь с ограниченными правами. Эксплуатация — самая слабая из трёх фаз: у AI нет рантайм-контекста.



Одно и то же внедрение AI даёт обе половины: +7,5% к качеству документации и –7,2% к стабильности доставки (DORA 2024 ^[1]); со стабильностью связь негативна второй год подряд (DORA 2025 ^[2]).

Какая из половин перевесит — решает зрелость конвейера, а не выбор модели.

Провал этой фазы — масштабировать AI на незрелый конвейер: вырастет и скорость, и нестабильность. Хайп: «AI-CD/ops-продукт как замена человека».

Множитель — это объём: зрелый гейт умножает проверенное, отсутствующий гейт умножает небезопасное



Где AI-копайлот работает

BT Group ^[1]

MTTR ~2ч → 85с (~97% сокращения)

MTTR (mean time to repair) — среднее время восстановления после сбоя

Microsoft Azure «Triangle» ^[2]

time-to-engage –91%, триаж 97%

time-to-engage — время до подключения дежурного инженера

Что умножает AI: корреляцию алертов и шаги по заранее написанному runbook.

Где гейт: runbook (готовый сценарий устранения) задаёт допустимые действия, телеметрия показывает результат каждого шага. Умножается уже проверенная работа — поверх УЖЕ зрелой практики эксплуатации (SRE).



Тот же множитель, обратная сторона

AI-генерируемый IaC (инфраструктура-как-код) ^[3]

~55% безопасны по умолчанию

доля задач, где сгенерированный Terraform / Kubernetes-код безопасен «из коробки», без правок человека; за 2 года цифра почти не сдвинулась. Синтаксическая корректность >95%.

Отдельный бенчмарк 2026 года, другое измерение: 8,4% на задачах с проверкой безопасности

Что умножает AI: генерацию конфигураций — в разы больше артефактов и быстрее.

Гейта нет: конвейер проверяет синтаксис, а не безопасность. Больше конфигураций → меньше внимания на каждую → небезопасное доезжает до прода.

Проблема не в том, что «выкаток стало больше», а в том, ЧТО именно умножается: VT умножает проверенный шаг, генерация IaC — непроверенный артефакт. Множитель один и тот же, разная только зрелость гейта.

Документация — единственный чистый плюс AI, но и у него есть парная цена



Светлый пяточок

Единственная фаза с чистым положительным системным эффектом AI. Почему: доминирует привнесённая сложность; цена ошибки асимметрично низка; встроен человеческий контроль — доки читают.

DORA 2024 ^[1]: +7,5% к качеству документации. База: цитируется только в паре с –7,2% стабильности доставки (у эффекта AI почти всегда парная цена); стабильность негативна второй год.



Когнитивный долг (Radar, Hold)

генерация документации обгоняет понимание: текста много, понимания меньше. Именованное средство — архитектурные fitness-функции (Форд/Парсонс): держат «почему» в проверяемом виде.



Онбординг-доки галлюцинируют настройку / развёртывание

Böckeler ^[2]: «AI не может волшебным образом заменить хорошо документированную и автоматизированную настройку».

Вторично: Confluence AI · AWS Q /doc · JetBrains KDoc/Javadoc.

Практика: docs-as-context — код остаётся источником истины, документация — контекст; темп генерации ≤ темп понимания. Документация-как-контекст — да; документация-как-истина — нет.

Инструментарий документации: вход — код, выход — пересказ сказанного



SaaS-вендорский слой

Confluence AI (Atlassian Intelligence)

Суммаризация длинного треда в выжимку;
генерация/трансформация черновика из промпта; Q&A-поиск по
базе знаний (RAG-подобный паттерн поверх корпоративной базы).

AWS Q Developer /doc

Агент анализирует кодовую базу, а не пересказывает промпт.
Строит диаграммы инфраструктуры из IaC-файлов (Terraform/CDK)
— прямая связь с уже введённым принципом архитектура-как-код.
Замкнутый цикл: код изменился → предложен дифф в
документации.



Код-ориентированная альтернатива

Вместо отдельного SaaS — сам кодинг-агент через установленный
skill («Code Documentation Skill», «README Generator»): анализирует
структуру проекта, зависимости, код-паттерны, генерирует
README/ADR/inline-комментарии.



Честная оговорка

*community-паттерн, не единый официальный skill из репозитория
Anthropic. Онбординг-документация — хороший кандидат в skill по
уже введённым эвристикам: повторяющаяся инструкция + нужен
progressive disclosure.*

Вендор-специфичный слой — иллюстрация механики текущего 2026-стека, не рекомендация одного вендора: тот же паттерн доступен через skill поверх уже используемого агента, без SaaS.

РАЗДЕЛ 7

Обобщение — дисциплина по фазам

Мы прошли все фазы; теперь свернём их в рабочий аппарат: матрицу «фаза × ведущая практика × где человек обязателен», триангуляцию независимых измерений, компактный risk-triad и чек-лист «когда AI да, когда нет».



0

Введение

1

Требования

2

Архитектура

3

Реализация

4

Тестирование

5

Ревью+Безоп.

6

Доставка+

7

Обобщение

Матрица лекции: ведёт практика — вендор-колонки нет вовсе

Фаза	Ведущая практика	Режим отказа	Где человек обязателен
 Требования	spec-driven: спека до кода	prompt-and-pray; «спека=истина»	решить, что строить
 Архитектура	ADR + fitness + арх-как-код	отравленный контекст без управления	выбор развилок под компромисс
 Реализация	explore→plan→code→commit + харнес	70%-проблема; «почти правильный»	ревью diff + merge
 Тестирование	TDD: тест-как-спека + детерм. гейт	«all green» лжёт; coverage≠дефекты	что тест утверждает
 Ревью + Безоп.	fresh-context; least-priv+SAST	благодущие; ложная уверенность	второй проход + угрозы
 Доставка	headless + риск-калибр. гейт (DORA-first)	AI потребляет, не владеет	прод-гейт (жёсткий на необратимом)
 Эксплуатация	телеметрия + on-call	нет системного контекста	владение моделью системы
 Документация	docs-as-context (код=истина)	когнитивный долг; галлюцинации	темп ≤ темп понимания

Вендор-имена из §1–§6 сменяемы; практика, режим отказа и точка человека здесь устойчивы — держатся на характере сложности фазы ^[2]. Каждая клетка выведена из разобранного раздела, не назначена. ^[1]

[1] Google — DORA 2025 · [2] Anthropic — AI-Native SDLC playbook

Три независимых метода сходятся: индивидуальная выгода AI ≠ системное качество



DORA (n ≈ 5000, системный) ^[1]

~90% отчётов: throughput положительный, но связь AI со стабильностью негативна второй год подряд. Линза: «AI усиливает то, что уже есть».



GitClear — два замера ^[2]

211М строк (2020–24): рефакторинг ~25%→<10%, дубли 8,3%→12,3%. 623М изменений (2023–26): рефакторинг 21%→3,8% (–70%), дубли +81%, churn +15%. (Обе — корреляция, не RCT.)



METR (n = 16, эксперты, знакомый код) ^[3]

задачи с AI заняли +19% времени, а верили в ускорение (~–20%) = разрыв восприятия. (На незнакомом коде эффект иной.)



Сила — в сходимости независимых методов: у DORA, GitClear и METR разные слепые пятна, поэтому вероятность, что все три ошиблись одинаково, мала. Общий вывод надёжнее любого одиночного числа.

Вывод один: методика важнее инструмента. Практика — CI-гейт на дублирование и churn; измерять системный эффект, а не ощущение.

Масштаб внедрения — ещё не эффект: решает не бренд и не охват, а применённая дисциплина



Uber, 2026 ^[1]

Uber

Uber · Wikimedia

Масштаб внедрения

агентные практики 32% → 84% за месяц (рост в 2,6 раза) · 95% инженеров ежемесячно · 70% закоммиченного кода — от AI · \$500–2000 на инженера в месяц

Эффект: не прослеживается

«Трудно провести связь между растущим использованием Claude Code и инновациями, которые реально служат потребителю... этой связи пока просто нет»

— Эндрю Макдональд, президент и операционный директор Uber

Реакция: потолок \$1500 на сотрудника в месяц — постфактум, после того как годовой бюджет сгорел за 4 месяца.



AWS Kiro — тот же продукт, два регистра ^[2]

aws

AWS · Wikimedia

Успех:

фарма и биотех (life sciences) — спека-first дисциплина, гейты проверяемости заранее → продакшен за 3 недели (тот же кейс, что уже был в начале лекции).

Провал:

Kiro-инцидент, декабрь 2025 — агент автономно снёс и пересобрал окружение без одобрения → многочасовой простой.

Разница — не бренд (продукт один и тот же), а применённая или пропущенная дисциплина: здесь исход измерим в обе стороны — в отличие от масштаба без критерия слева.

Рост внедрения сам по себе не является результатом: 2,6x за месяц без заранее заданного критерия дали рост расходов и неподтверждённый эффект. Решение о масштабе AI — измеримое инженерное решение с критерием, заданным заранее, а не культурная инерция.

Вопрос не «AI или нет», а какой уровень автономии: вероятность × влияние × обнаружимость

AI в том или ином режиме применяется почти всегда — триада отвечает не «да / нет», а какой потолок автономии допустим и чем он оплачен.

1. Вероятность ошибки

низкая → высокая

растёт с незнакомостью задачи (ось SWE-bench Pro)

2. Влияние ошибки

низкое → высокое

необратимость, безопасность, деньги, данные

3. Обнаружимость

низкая → высокая

есть ли тест-оракул, SAST, ревью, которые поймут ошибку

Где потолок автономии самый высокий

Полный **vibe-coding** — только при сочетании **низкая × низкая × высокая**: **низкая вероятность ошибки, низкое влияние, высокая обнаружимость**. Любая другая комбинация — не запрет AI, а потолок ниже и обязательные меры. Оси перемножаются, не складываются.

Чем оплачивается более высокий уровень:

- влияние ↑ → жёсткий человеческий гейт
- обнаружимость ↓ → машинный оракул
- вероятность ↑ → построчное senior-ревью

Böckeler [1]: «использование генеративного AI — постоянная оценка риска»: решение принимается не один раз и не про инструмент целиком, а на каждой задаче. Провал — **vibe-coding «по ощущению»:** ни одна из трёх осей не посчитана. В нём сходятся все кейсы лекции: **Replit (влияние ↑), curl-slop (обнаружимость ↓), уязвимый код (вероятность ↑).**

Восемь вопросов — не «AI или нет», а как, где и с каким контролем его применить

- ✓ 1. Какая это фаза жизненного цикла? Она задаёт режим отказа
- ✓ 2. Что здесь решается детерминированно? Эту часть пишет обычный код, AI — на разбор и проверку
- ✓ 3. Существенная или привнесённая сложность? Существенная — решает человек, AI на периферии
- ✓ **4. Обратимо ли последствие? Необратимое → потолок автономии вниз, жёсткий гейт — ВЕТО-ось**
- ✓ 5. Есть ли машинный оракул (тест, SAST, прогон)? Нет → сначала оракул, потом автономия
- ✓ 6. Затронуты секреты / недоверенный контент? Да → минимум прав и изоляция
- ✓ 7. Кто ревьюит и кто мержит? Слияние и ответственность — всегда человек
- ✓ 8. Цель — артефакт или навык? Навык → генерацию не делегировать



Anthropic, Shen & Tamkin 2026 (RCT, n=52, освоение незнакомой библиотеки) ^[1]: группа с AI на квизе 50% против 67% без AI (~–17 п.п.). Кто делегировал генерацию — просел; кто спрашивал концепции («как работает, почему») — деградации нет. Ускорение статистически не значимо.

Когда цель — навык, пишете вы; AI объясняет и проверяет.

Чек-лист не решает «применять AI или нет» — он выдаёт режим: для подходящей задачи приведёт к высокой автономии, для неподходящей — опустит потолок и назовёт обязательные меры. Это распределение бремени доказательства, а не «всегда меньше AI». Необратимость и влияние — вето-ось.

AI меняет цену написания кода — не цену понимания и ответственности

AI меняет цену написания кода, но не цену понимания, что строить и кто за это отвечает. Он касается каждой фазы по-разному — и надёжность даёт не инструмент, а дисциплина по фазам.

Метод переносится на все отрасли (не список инструментов):

- 1 разложить деятельность на фазы
- 2 спросить: привнесённая или существенная сложность
- 3 потребовать базу для каждого числа и системный эффект
- 4 отделить устойчивый паттерн от вендор-хайпа пятью вопросами

Вопросы?

